DBMS PROJECT REPORT

ResolveIQ: Customer Complaint Matching and Resolution System

Database Management Systems – Experiential Learning Project

1. PROJECT TITLE

ResolveIQ: Customer Complaint Matching and Resolution System

ResolveIQ is a customer-support application that stores customer complaints in a relational database and uses semantic similarity search to identify related historical complaints. Support agents review suggested matches, accept or reject them, communicate progress and suggestions to customers, and record complaint resolutions. Customers can track the assigned agent, department, complaint status, AI match decisions, and agent updates.

2. STUDENT SRNs AND NAMES

3. ABSTRACT

ResolveIQ supports customer complaint handling by combining structured data management with semantic search. MySQL stores information about roles, user accounts, customers, products, complaint categories, priorities, statuses, support agents, departments, assignments, resolutions, AI-generated matches, and feedback. When a complaint is submitted, the backend creates an embedding using the all-MiniLM-L6-v2 sentence-transformer model and searches a Qdrant vector collection for semantically similar historical complaints. The model produces 384-dimensional vectors and the application uses cosine similarity. Matches meeting the configured threshold are recorded in MySQL with an initial Suggested status. The Next.js interface provides customer complaint submission, a support-agent dashboard, and a customer tracking portal. Agents can review historical matches, accept or reject suggestions, and post suggestions, progress updates, and resolution messages. Customers can view their complaint status, assigned agent and department, AI match decisions, and update history. The project demonstrates practical integration of relational SQL and vector database paradigms.

4. TECHNOLOGY STACK

5. SYSTEM OVERVIEW AND WORKFLOW

1. A customer selects a demo customer profile and submits a complaint with relevant details.

2. FastAPI stores the complaint in MySQL and creates its text embedding.

3. Qdrant searches for semantically similar historical complaint vectors.

4. The backend records qualifying matches in MySQL for agent review.

5. A support agent reviews matches and marks them Accepted or Rejected.

6. The agent posts a Suggestion, Progress Update, or Resolution, optionally updating the complaint status.

7. The customer portal retrieves and displays the assigned agent, department, complaint status, AI decisions, and update history.

The customer profile selector is a demonstration mechanism. Production deployment should use authenticated customer accounts and enforce ownership checks on customer-specific endpoints.

6. SQL – DATABASE COMMANDS

The relational database is named resolveiq_db. The original schema contains ROLE, USER_ACCOUNT, CUSTOMER, SUPPORT_DEPARTMENT, SUPPORT_AGENT, PRODUCT, COMPLAINT_CATEGORY, PRIORITY, COMPLAINT_STATUS, COMPLAINT, COMPLAINT_ASSIGNMENT, RESOLUTION, COMPLAINT_RESOLUTION, COMPLAINT_MATCH, and COMPLAINT_FEEDBACK. The customer-tracking extension adds COMPLAINT_AGENT_UPDATE.

6.1 Database selection and inspection

```
CREATE DATABASE IF NOT EXISTS resolveiq_db;
USE resolveiq_db;
SHOW TABLES;
DESCRIBE CUSTOMER;
DESCRIBE COMPLAINT;
DESCRIBE COMPLAINT_ASSIGNMENT;
DESCRIBE COMPLAINT_MATCH;
DESCRIBE COMPLAINT_AGENT_UPDATE;
```

6.2 Customer complaint details

```
SELECT c.complaint_id, cu.customer_name, c.subject,
    c.description, p.priority_name, cs.status_name, c.created_at
FROM COMPLAINT c
JOIN CUSTOMER cu ON c.customer_id = cu.customer_id
JOIN PRIORITY p ON c.priority_id = p.priority_id
JOIN COMPLAINT_STATUS cs ON c.status_id = cs.status_id
ORDER BY c.created_at DESC;
```

6.3 Complaint count per customer

```
SELECT cu.customer_name, COUNT(c.complaint_id) AS total_complaints
FROM CUSTOMER cu
LEFT JOIN COMPLAINT c ON cu.customer_id = c.customer_id
GROUP BY cu.customer_id, cu.customer_name
ORDER BY total_complaints DESC;
```

6.4 Active complaints assigned to each agent

```
SELECT sa.agent_name, COUNT(ca.complaint_id) AS assigned_complaints
FROM SUPPORT_AGENT sa
JOIN COMPLAINT_ASSIGNMENT ca ON sa.agent_id = ca.agent_id
WHERE ca.assignment_status = 'Active'
GROUP BY sa.agent_id, sa.agent_name
ORDER BY assigned_complaints DESC;
```

6.5 Retrieve AI matches and review status

```
SELECT match_id, complaint_id, matched_complaint_id,
    similarity_score, match_status
FROM COMPLAINT_MATCH
ORDER BY similarity_score DESC;
```

6.6 Accept or reject an AI match

```
UPDATE COMPLAINT_MATCH
SET match_status = 'Accepted'
WHERE match_id = 1;

UPDATE COMPLAINT_MATCH
SET match_status = 'Rejected'
WHERE match_id = 2;
```

6.7 Aggregate AI matches by review status

```
SELECT match_status, COUNT(*) AS total_matches,
    AVG(similarity_score) AS average_similarity
FROM COMPLAINT_MATCH
GROUP BY match_status;
```

6.8 Create customer agent-update history table

```
CREATE TABLE IF NOT EXISTS COMPLAINT_AGENT_UPDATE (
  update_id INT NOT NULL AUTO_INCREMENT,
  complaint_id INT NOT NULL,
  agent_id INT NOT NULL,
  update_type VARCHAR(30) NOT NULL,
  update_message TEXT NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (update_id),
  KEY idx_agent_update_complaint (complaint_id, created_at),
```

```
    KEY idx_agent_update_agent (agent_id),
    CONSTRAINT fk_agent_update_complaint FOREIGN KEY (complaint_id)
      REFERENCES COMPLAINT(complaint_id)
      ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_agent_update_agent FOREIGN KEY (agent_id)
      REFERENCES SUPPORT_AGENT(agent_id)
      ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_agent_update_type CHECK (
      update_type IN ('Suggestion', 'Progress Update', 'Resolution')
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

SHOW CREATE TABLE COMPLAINT_AGENT_UPDATE;
```

6.9 Retrieve customer profiles

```
SELECT customer_id, customer_name
FROM CUSTOMER
ORDER BY customer_name;
```

6.10 Retrieve complaints for one customer

```
SELECT c.complaint_id, c.subject, c.description, c.created_at,
       cs.status_name, p.priority_name,
       sa.agent_id, sa.agent_name, sd.department_name,
       ca.assignment_status
FROM COMPLAINT c
JOIN COMPLAINT_STATUS cs ON cs.status_id = c.status_id
JOIN PRIORITY p ON p.priority_id = c.priority_id
LEFT JOIN COMPLAINT_ASSIGNMENT ca
  ON ca.complaint_id = c.complaint_id
 AND ca.assignment_status = 'Active'
LEFT JOIN SUPPORT_AGENT sa ON sa.agent_id = ca.agent_id
LEFT JOIN SUPPORT_DEPARTMENT sd ON sd.department_id = sa.department_id
WHERE c.customer_id = 1
ORDER BY c.created_at DESC, c.complaint_id DESC;
```

6.11 Retrieve agent updates for a complaint

```
SELECT u.update_id, u.complaint_id, u.update_type,
       u.update_message, u.created_at, sa.agent_name,
       sd.department_name
FROM COMPLAINT_AGENT_UPDATE u
JOIN SUPPORT_AGENT sa ON sa.agent_id = u.agent_id
JOIN SUPPORT_DEPARTMENT sd ON sd.department_id = sa.department_id
WHERE u.complaint_id = 2
ORDER BY u.created_at DESC, u.update_id DESC;
```

6.12 Insert an agent update

```
INSERT INTO COMPLAINT_AGENT_UPDATE
```

```
    (complaint_id, agent_id, update_type, update_message)
VALUES
    (2, 2, 'Progress Update',
     'We are reviewing the transaction details.');
```

6.13 Update complaint status to Resolved

```
UPDATE COMPLAINT
SET status_id = (
    SELECT status_id FROM COMPLAINT_STATUS
    WHERE status_name = 'Resolved'
),
resolved_at = CURRENT_TIMESTAMP
WHERE complaint_id = 2;
```

6.14 Verify saved updates and complaint status

```
SELECT c.complaint_id, c.subject, cs.status_name, c.resolved_at
FROM COMPLAINT c
JOIN COMPLAINT_STATUS cs ON cs.status_id = c.status_id
WHERE c.complaint_id = 2;

SELECT update_type, update_message, created_at
FROM COMPLAINT_AGENT_UPDATE
WHERE complaint_id = 2
ORDER BY created_at DESC;
```

Additional SQL concepts demonstrated during development include primary and foreign keys, joins, GROUP BY, HAVING, ORDER BY, aggregate functions, subqueries, indexes, transaction handling, a complaint-details view, a trigger for resolved timestamps, and query-plan inspection. The SQL snippets above are executable examples; IDs should be changed to match the records in the local database.

7. NOSQL / VECTOR DATABASE COMMANDS

ResolveIQ uses Qdrant in local mode. The all-MiniLM-L6-v2 model encodes complaint subject and description text into 384-dimensional vectors. Qdrant stores each vector with a payload containing complaint ID, subject, and description. Similarity search uses cosine similarity, and the backend applies the configured MATCH_THRESHOLD (0.50 in the documented implementation).

7.1 Install vector dependencies

```
python -m pip install qdrant-client sentence-transformers
```

7.2 Initialize Qdrant and create collection

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams

qdrant = QdrantClient(path="./qdrant_storage")
COLLECTION_NAME = "complaints"

if not qdrant.collection_exists(COLLECTION_NAME):
```

```
qdrant.create_collection(
    collection_name=COLLECTION_NAME,
    vectors_config=VectorParams(size=384, distance=Distance.COSINE)
)
```

7.3 Load embedding model and encode complaint

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
text = f"{subject}. {description}"
vector = model.encode(text).tolist()
print(len(vector))  # Expected: 384
```

7.4 Upsert complaint vector and payload

```
from qdrant_client.models import PointStruct

qdrant.upsert(
    collection_name=COLLECTION_NAME,
    points=[
        PointStruct(
            id=complaint_id,
            vector=vector,
            payload={
                "complaint_id": complaint_id,
                "subject": subject,
                "description": description
            }
        )
    ]
)
```

7.5 Search similar complaints

```
results = qdrant.query_points(
    collection_name=COLLECTION_NAME,
    query=vector,
    limit=3
).points

for result in results:
    print(result.payload, result.score)
```

7.6 Inspect collection

```
print(qdrant.get_collections())
print(qdrant.get_collection("complaints"))
```

The local Qdrant storage directory is qdrant_storage. The local-mode database should be accessed by only one backend process at a time to avoid file-lock conflicts.

8. FASTAPI BACKEND – INSTALLATION, RUN AND API COMMANDS

8.1 Create and activate Python virtual environment

```
cd ~/Downloads/resolveiq-vector
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
python -m pip install fastapi uvicorn mysql-connector-python python-dotenv
python -m pip install qdrant-client sentence-transformers
```

8.2 Configure environment variables

Create a local .env file in resolveiq-vector and configure the MySQL connection values required by api.py. Do not include database passwords in the submitted report or public repository.

```
# Example variable names; use the exact names read by api.py
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_local_password
DB_NAME=resolveiq_db
```

8.3 Run backend API

```
cd ~/Downloads/resolveiq-vector
source venv/bin/activate
python -m uvicorn api:app --reload --port 8000
```

8.4 Test backend health and interactive API documentation

```
curl http://127.0.0.1:8000/health
# Open API documentation in a browser:
http://127.0.0.1:8000/docs
```

8.5 Customer and tracking API endpoints

8.6 Test customer tracking endpoints

```
curl http://127.0.0.1:8000/customers
curl http://127.0.0.1:8000/customers/1/complaints
curl http://127.0.0.1:8000/complaints/2/updates
```

8.7 Example agent update request

```
curl -X POST http://127.0.0.1:8000/complaints/2/updates \
  -H "Content-Type: application/json" \
  -d '{
    "agent_id": 2,
    "update_type": "Progress Update",
    "update_message": "We are reviewing the transaction details.",
    "status_name": "In Progress"
  }'
```

The POST update endpoint accepts update types Suggestion, Progress Update, and Resolution. It verifies that the specified agent has an active assignment to the complaint, saves the update in COMPLAINT_AGENT_UPDATE, and changes the complaint status when status_name is supplied. Resolved and Closed statuses set resolved_at; other allowed statuses clear it.

9. NEXT.JS FRONTEND – INSTALLATION AND PROJECT TREE

9.1 Install and run frontend

```
cd ~/Downloads/resolveiq-web
npm install
npm run dev
# Open http://localhost:3000
```

9.2 Production build verification

```
cd ~/Downloads/resolveiq-web
npm run build
```

The updated frontend production build completed successfully, including compilation, TypeScript validation, page-data collection, and static page generation.

9.3 Project tree structure

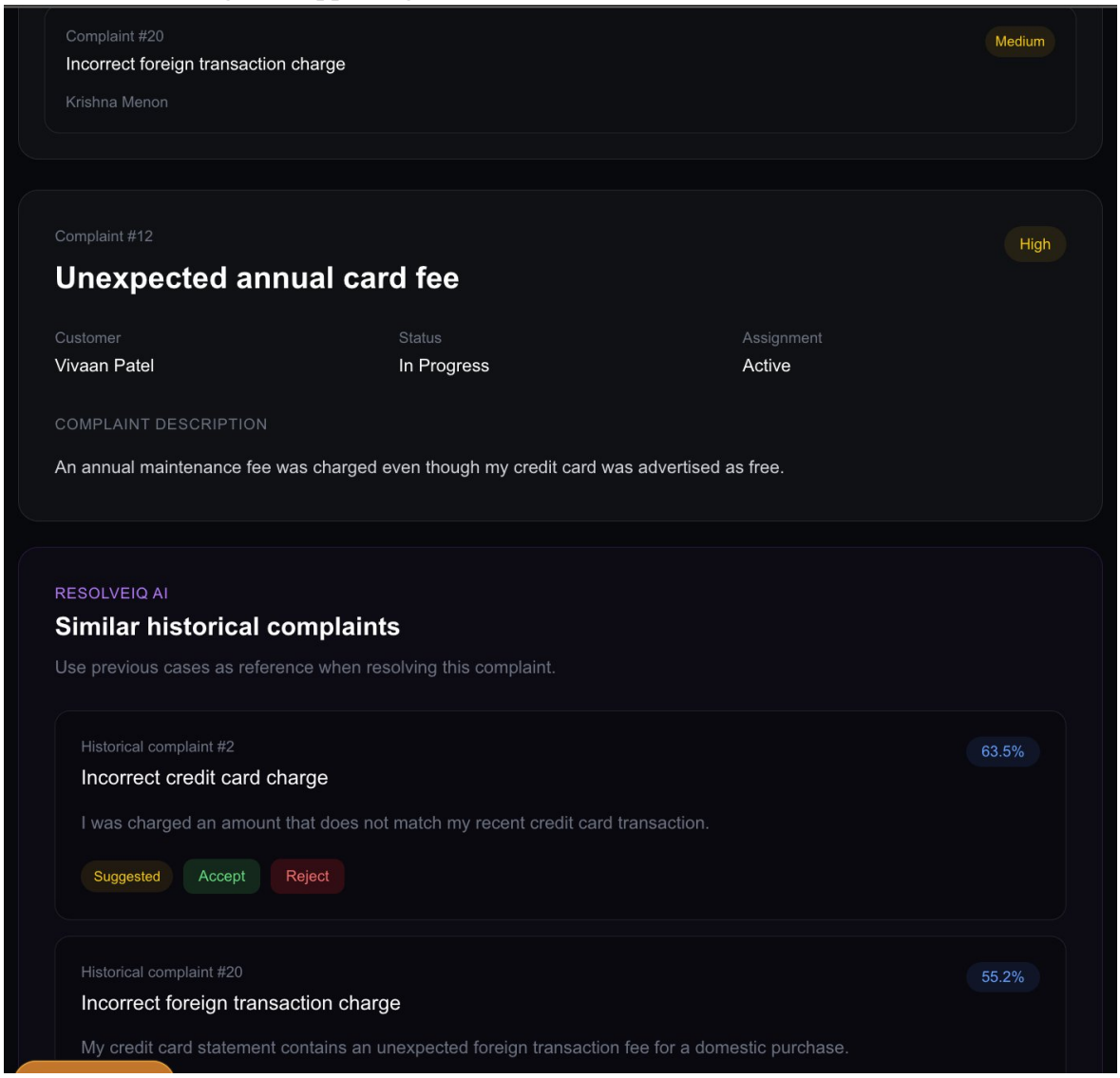
```
ResolveIQ/
|
+-- resolveiq-web/
|   +-- app/
|   |   +-- page.tsx
|   |   +-- layout.tsx
|   |   +-- globals.css
```

```
|  +-- public/
|  +-- package.json
|  +-- tsconfig.json
|  +-- next.config.ts
|
+-- resolveiq-vector/
|  +-- api.py
|  +-- setup_qdrant.py
|  +-- requirements.txt
|  +-- .env              (local secrets; do not submit)
|  +-- qdrant_storage/   (local Qdrant data)
|  +-- venv/             (Python environment; do not submit)
|
+-- resolveiq_db.sql     (MySQL database export)
```

10. SAMPLE SCREENSHOTS OF APPLICATION AND DEMO

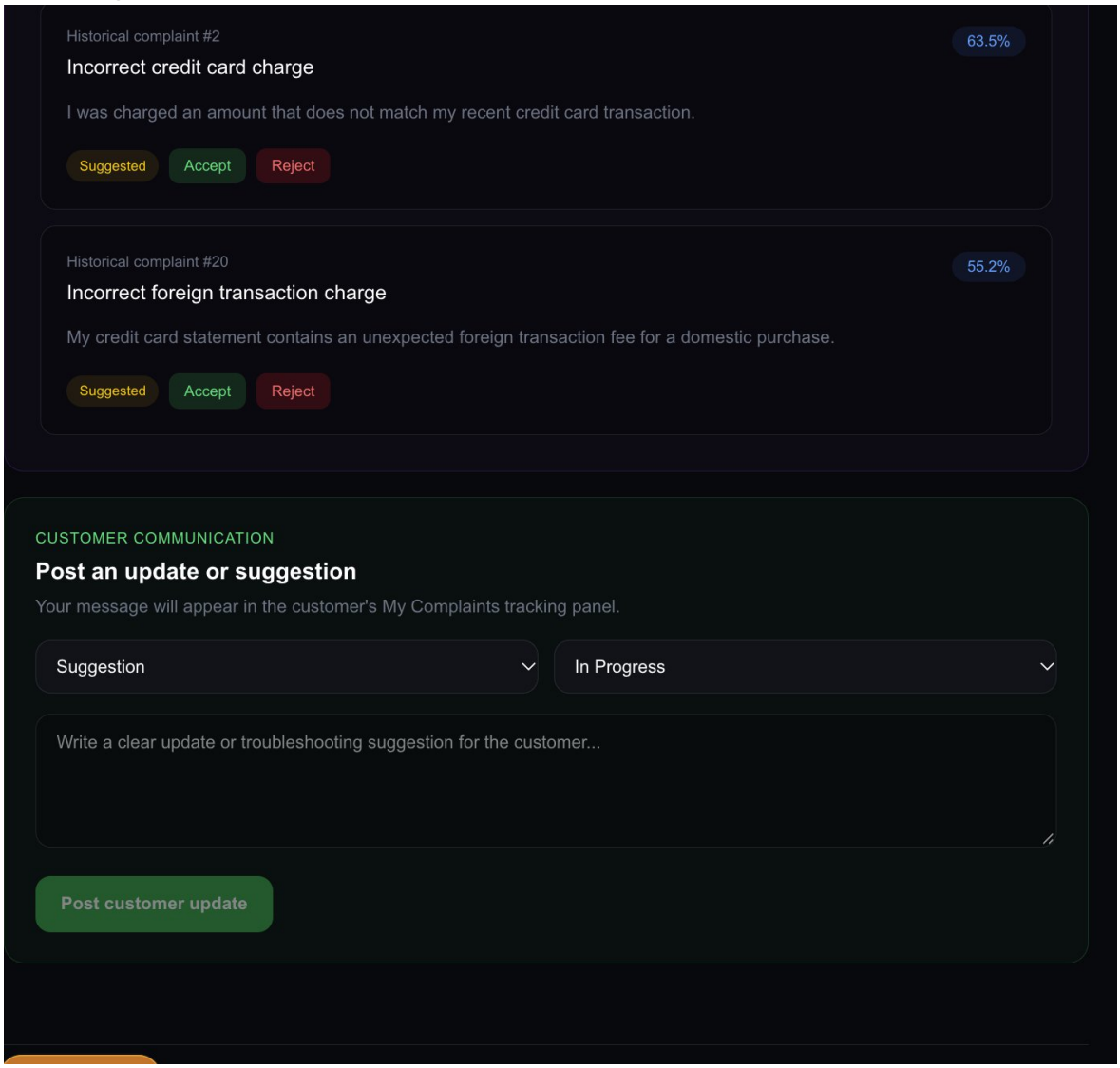
The following screenshots were captured from the running ResolveIQ application during the customer-tracking feature verification. They document AI matching, agent communication, customer-visible progress updates, and the resolved complaint state.

Screenshot 1 – AI matching and support-agent dashboard



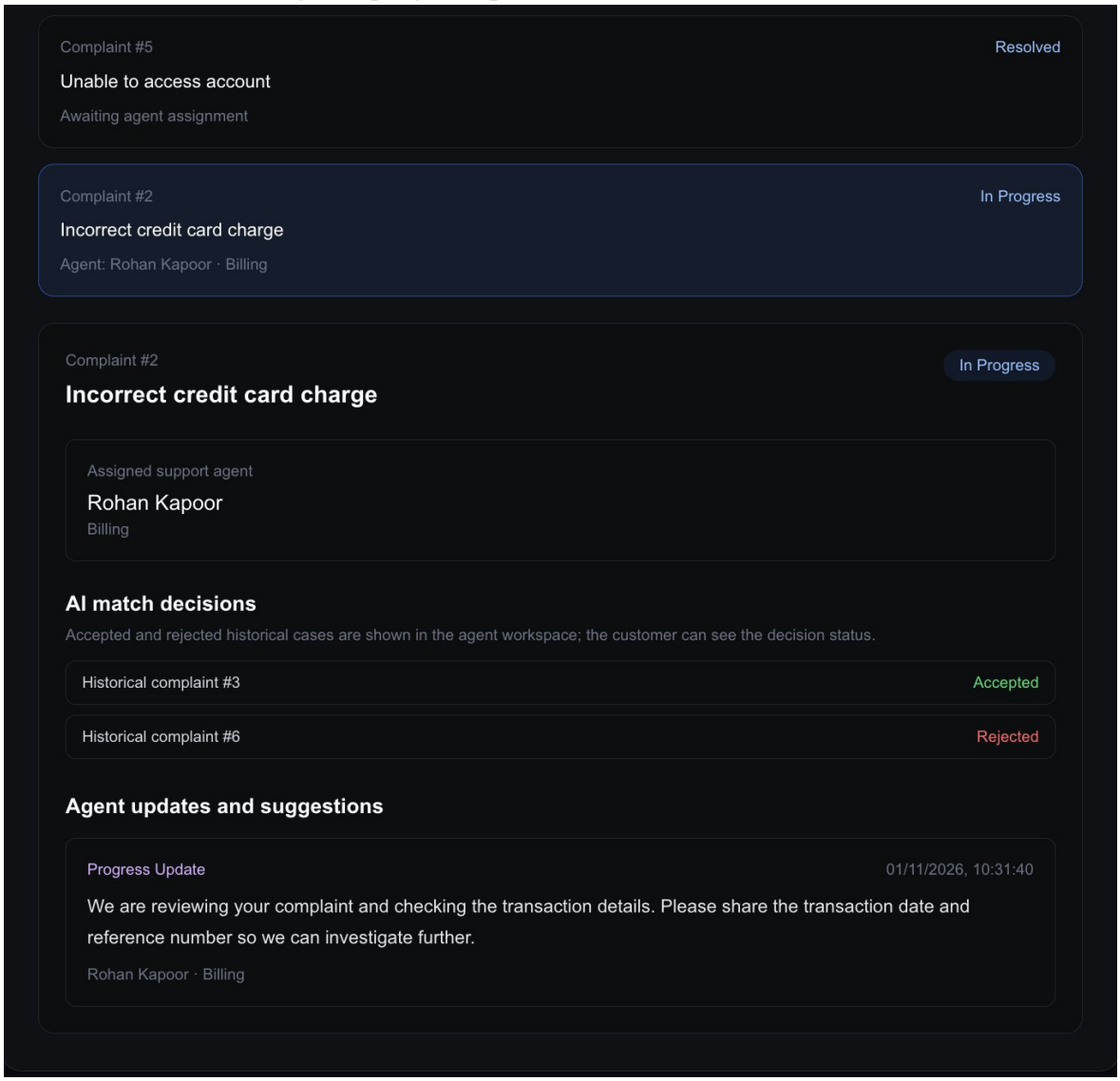
Caption: Support agent workspace showing assigned complaint details, AI-generated historical complaint matches, similarity percentages, and Accept/Reject controls.

Screenshot 2 – Agent customer communication



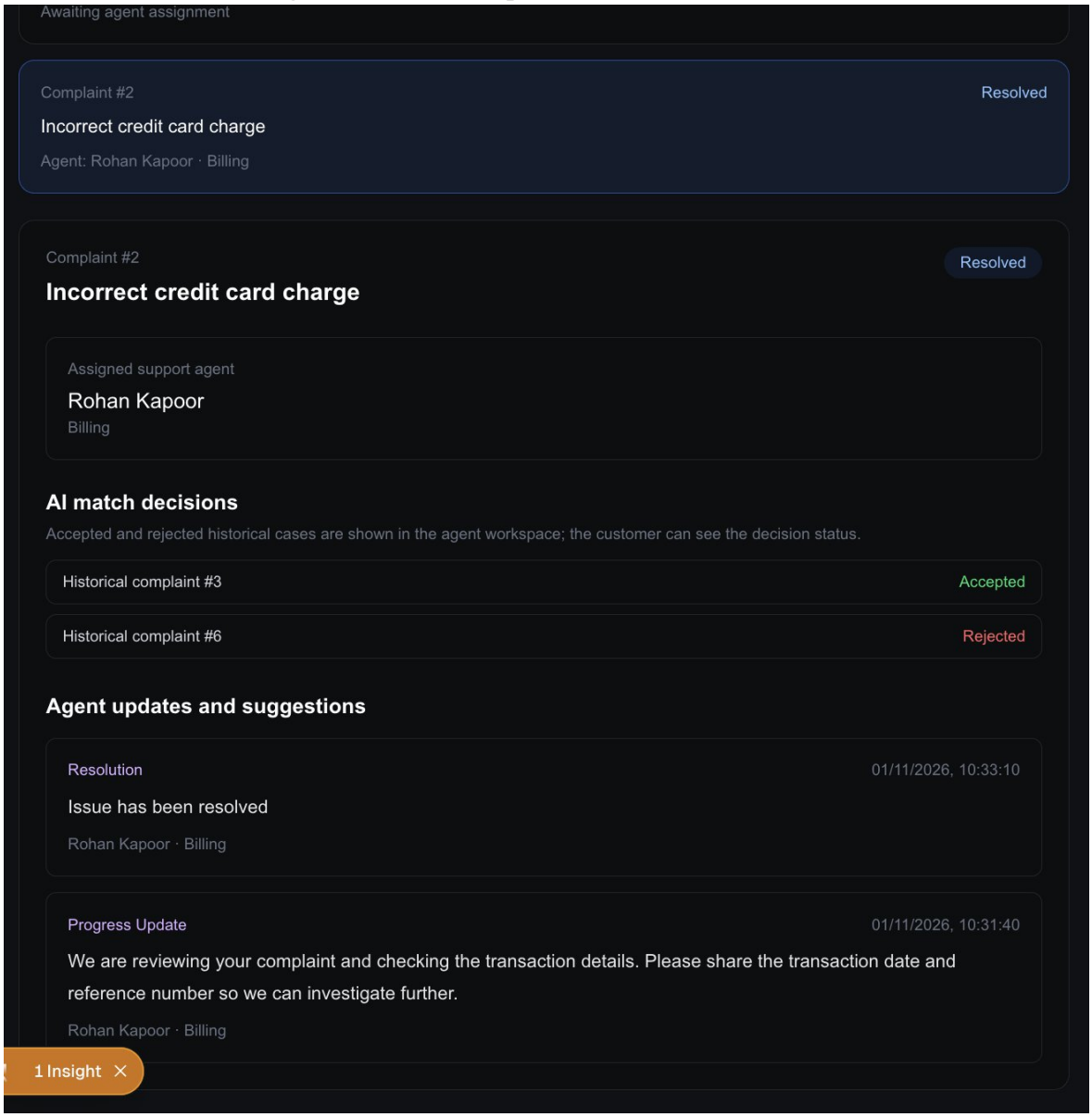
Caption: Agent communication panel with update type, complaint status selector, message field, and Post customer update action.

Screenshot 3 – Customer tracking with progress update



Caption: Customer portal showing assigned agent and department, AI match decisions, and the agent's progress update with timestamp.

Screenshot 4 – Customer tracking with resolved complaint



Caption: Customer portal showing complaint status Resolved, assigned agent, AI match decisions, and the resolution message in the update history.

11. EVALUATION RUBRICS – COMPLETION STATUS

The customer profile selector is currently a demo profile selector and should not be described as production authentication. The application build passed successfully after the customer-loading logic was added. The customer and agent tracking workflow was manually tested using the screenshots included above.

12. APPLICATION FEATURES

ResolveIQ provides a customer complaint submission portal with demo customer-profile selection, MySQL-based complaint storage, and AI-powered semantic matching of new complaints against historical cases using the all-MiniLM-L6-v2 embedding model and Qdrant vector search. Support agents can view assigned complaints, inspect similarity scores, accept or reject suggested matches, and communicate with customers through suggestions, progress updates, and resolution messages. Agents can also update complaint statuses, while customers can track each complaint, view the assigned agent and department, see AI match decisions, and review timestamped update history. The application uses a Next.js frontend, a FastAPI backend, MySQL relational storage, and a local Qdrant vector database.

13. OFFICIAL PROJECT REQUIREMENTS (1–13) – COMPLETION AND EVIDENCE

The following table maps the first thirteen project requirements to the corresponding implementation areas and evidence in ResolveIQ. Completion status reflects the project scope and the implementation demonstrated in the application and database.

No.	Project requirement	Status	Where implemented / evidence
1	Unique project title	**Met**	Project title page and report title: “ResolveIQ: Customer Complaint Matching and Resolution System”. The team has confirmed title uniqueness with the instructor/section.
2	Team formation and defined member responsibilities	**Met**	Student details section identifies Suravi (PES1UG24AM295) and Sriviswaja (PES1UG24AM288). Both members contributed to database and application work
3	Use SQL with a NoSQL or vector database	**Met**	MySQL database `resolveiq_db` stores structured records; Qdrant local collection `complaints` stores complaint embeddings for vector similarity search.
4	Database design, ER diagram and relational schema	**Met**	Database Design section and submitted ER diagram/relational schema. The sche includes users, roles, customers, agents, departments, products, complaints, assignments, matches, resolutions and feedback/update records.
5	SQL DDL commands	**Met**	SQL – Database Commands section: CREATE DATABASE, CREATE TABLE, ke constraints and the COMPLAINT_AGENT_UPDATE table definition; schema can inspected with DESCRIBE and SHOW CREATE TABLE.
6	SQL DML commands and queries	**Met**	SQL query examples for INSERT, SELECT, UPDATE, JOIN, GROUP BY, aggreg functions, filtering, ordering, complaint assignment, AI match review and custome tracking.
7	At least four advanced database concepts	**Met**	Implemented concepts include the `complaint_details` view, `set_resolved_time` trigger, indexes on frequently queried columns, PRIMARY KEY / FOREIGN KEY / UNIQUE / CHECK constraints, the `GetComplaintCountByStatus` stored procedu and the `GetCustomerComplaintCount` stored function. Backend database operat also use COMMIT and ROLLBACK transaction handling.
8	Vector embeddings and semantic search	**Met**	Backend embedding generation uses `all-MiniLM-L6-v2` to create 384-dimensiona complaint vectors. Qdrant stores vectors and performs cosine-similarity search; qualifying matches are recorded in MySQL.
9	Next.js web application	**Met**	Next.js frontend in `resolveiq-web/app/page.tsx` provides customer complaint submission and tracking, and a support-agent dashboard with complaint review ar communication. Production build was tested.
10	Integration between database and application	**Met**	Next.js communicates with FastAPI; FastAPI connects to MySQL and Qdrant. The workflow covers complaint submission, embedding/search, match review, agent updates and customer tracking.
11	Authentication and authorization / role-based access	**Met for role-based workflow demonstration**	`ROLE` and `USER_ACCOUNT` link accounts to roles through `role_id`; Custome and Support Agent workflows are demonstrated separately. The current selectors demo profile selectors, not production login authentication; password-hash storag exists in the schema, but password hashing/login verification and API-level authorization were not verified in the inspected code.
12	Data validation and integrity	**Met**	Pydantic request models validate complaint and agent-update inputs, including required text, length limits, positive identifiers and permitted values. MySQL enfor primary keys, foreign keys, unique constraints, CHECK constraints and referential integrity.
13	Application and database security	**Met within demonstrated project scope**	Security-related implementation includes environment-based database credentials restricted CORS origin, request validation, database constraints, account-status v and role-linked account records. The `.env` file is local and must not be committed included in the submitted report. Production authentication and authorization woul require additional implementation.

Note on Requirement 11: the project demonstrates role-linked account data and separate customer/agent workflows for the academic demo. The current profile selectors do not authenticate a real person. This distinction is stated explicitly so the report does not overclaim production login security.

14. ADDITIONAL ADVANCED DATABASE OBJECTS

In addition to the view, trigger, indexes and constraints, ResolveIQ includes a stored procedure and a stored function to demonstrate reusable database-side logic.

14.1 Stored Procedure – GetComplaintCountByStatus

This procedure accepts a complaint status name and returns the number of complaints currently associated with that status. It uses a LEFT JOIN between COMPLAINT_STATUS and COMPLAINT.

```
DELIMITER //

CREATE PROCEDURE GetComplaintCountByStatus(
    IN p_status_name VARCHAR(50)
)
BEGIN
    SELECT
        cs.status_name,
        COUNT(c.complaint_id) AS total_complaints
    FROM COMPLAINT_STATUS cs
    LEFT JOIN COMPLAINT c
        ON cs.status_id = c.status_id
    WHERE cs.status_name = p_status_name
    GROUP BY cs.status_name;
END //

DELIMITER ;

-- Example call
CALL GetComplaintCountByStatus('Resolved');
```

14.2 Stored Function – GetCustomerComplaintCount

This function accepts a customer ID and returns the total number of complaints submitted by that customer. It returns an integer and can be called from a SELECT statement.

```
DELIMITER //

CREATE FUNCTION GetCustomerComplaintCount(
    p_customer_id INT
)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE total_count INT;

    SELECT COUNT(*)
    INTO total_count
    FROM COMPLAINT
    WHERE customer_id = p_customer_id;

    RETURN total_count;
END //

DELIMITER ;

-- Example function call
SELECT GetCustomerComplaintCount(1) AS total_complaints;
```

Viva distinction: A stored procedure is invoked using CALL and can execute a sequence of SQL statements. A stored function returns a value and can be used within SQL expressions, including SELECT statements.